Merge Sort & Multithreaded Merge Sort: Performance Comparison

LP-III : Design & Analysis of Algorithms (410241) - Mini Project (Topic 2)

Name: _____ Roll No.: _____ Class/Div: _____

1. Problem Statement

Given an array of **n** numbers, sort it in ascending order using **Merge Sort**. Implement two versions of the same algorithm:

- **Normal Merge Sort:** runs on a single thread. It divides the array into two halves, sorts each half one after the other, and merges them.
- **Multithreaded Merge Sort:** sorts the two halves at the same time on different CPU cores using threads, and then merges them.

Compare both versions on **execution time** for different input sizes, and analyse their performance for **best-case** and **worst-case** inputs.

2. Objectives

- To understand the Divide and Conquer strategy using Merge Sort.
- To implement the normal (single-threaded) Merge Sort in C++.
- To implement a multithreaded Merge Sort in C++ using `std::thread`.
- To build best-case, worst-case, reverse-sorted and random inputs and count the comparisons made.
- To measure execution time of both versions for small and large inputs and calculate the speed-up.
- To study how the number of threads affects the speed-up.
- To analyse the time and space complexity of both versions.

3. Software & Hardware Requirements

Software

- Operating System: Windows 10/11 or Linux (Ubuntu)
- Language: C++ (C++11 or higher, needed for `std::thread`)
- Compiler: g++ (GCC) or MinGW-w64 with POSIX threads (compile with `-pthread`)
- Editor/IDE: VS Code, Code::Blocks or Dev-C++

Hardware

- Processor: multi-core CPU (Intel Core i3 or higher, at least 2 cores; more cores give more speed-up)
- RAM: 4 GB minimum (the 50 lakh element test needs about 60 MB)
- Storage: 500 MB free space

Results in this report were taken on: Intel Core i5-1235U (10 cores: 2 performance + 8 efficient, 12 hardware threads), 16 GB RAM, Windows 11, g++ 13.2 (MSYS2), compiler flag -O2.

4. Theory / Concept

4.1 Divide and Conquer

Divide and Conquer solves a big problem in three steps:

- **Divide:** break the problem into smaller sub-problems of the same type.
- **Conquer:** solve each sub-problem (recursively). Very small sub-problems are solved directly.
- **Combine:** join the answers of the sub-problems into the answer of the full problem.

4.2 Merge Sort

- **Divide:** split the array at the middle index m = (l + r) / 2 into a left half and a right half.
- **Conquer:** sort both halves by calling Merge Sort on them. An array of one element is already sorted.
- **Combine:** merge the two sorted halves into one sorted array.

Merge Sort is **stable** (equal elements keep their order), always takes **O(n log n)** time, and needs an extra array of size n for merging.

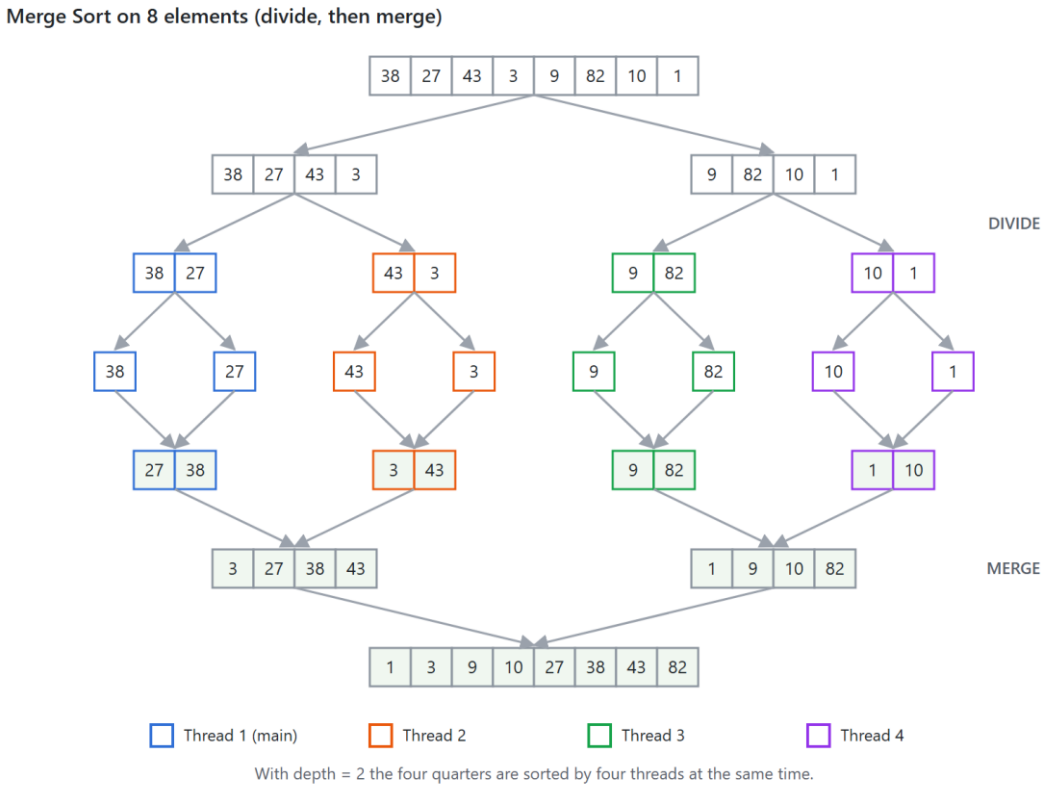


Figure 1: Merge Sort on [38, 27, 43, 3, 9, 82, 10, 1]. Colours show which thread sorts each quarter in the multithreaded version.

4.3 The Merge Step

Two pointers i and j start at the beginning of the left and right halves. The smaller of `left[i]` and `right[j]` is copied to the result and that pointer moves forward. When one half is empty, the rest of the other half is copied as it is. Each comparison places one element, so merging two halves of total size n needs **at most n − 1 comparisons** and at least min(size of left, size of right) comparisons.

4.4 Threads

- A **thread** is a small unit of execution inside a program. All threads of a program share the same memory, so they can work on the same array.

- In C++, `std::thread t(func, args...)` starts `func` on a new thread. `t.join()` makes the current thread wait until `t` finishes.
- If the computer has several cores, different threads run **truly in parallel** on different cores.
- Creating a thread has a fixed cost (roughly 20 to 100 microseconds). For small work, this cost is more than the time saved.

4.5 Multithreaded Merge Sort

- The two halves of the array do not overlap, so they can be sorted at the same time without any locks. This is why Merge Sort is easy to run in parallel.
- At each level the **left half is given to a new thread** and the **right half is sorted by the current thread**. The current thread then calls `join()` and merges.
- A **depth** limit stops creating threads after a few levels. With depth = 3 the program uses at most 2^3 = **8 threads**. Below that, normal recursion is used.
- A **THRESHOLD** (1000 elements) stops creating threads for small parts, because the thread start-up cost would be more than the work.

4.6 Best Case and Worst Case Inputs

Merge Sort always divides and merges the same way, so its time is O(n log n) in every case. Only the **number of comparisons in the merge step** changes:

- **Best case:** the array is already sorted. Every element of the left half is smaller than the right half, so after min(size) comparisons the left half is empty and the rest is copied. About $(n/2)\cdot\log_2 n$ comparisons.
- **Worst case:** the left and right halves alternate (like 1, 3, 5, 7 and 2, 4, 6, 8), so the merge compares until the very last element. Built in the program by "un-merging" a sorted array: elements at even positions go left and odd positions go right, recursively. About $n\cdot\log_2 n$ comparisons.
- **Reverse sorted:** every element of the right half is smaller, so it behaves like the best case (about $(n/2)\cdot\log_2 n$ comparisons).
- **Random:** close to the worst case (about $n\cdot\log_2 n - 1.25n$ comparisons).

4.7 Speed-up and Efficiency

Speed-up tells how many times faster the multithreaded version is. **Efficiency** tells how well each thread is used. If speed-up is less than 1, the multithreaded version is slower.

5. Algorithm & Flowchart

5.1 Merge (a, l, m, r)

1. Copy a[l..m] into array left and a[m+1..r] into array right.
2. Set i = 0, j = 0, k = l.
3. While i < size(left) and j < size(right):
    - If left[i] ≤ right[j], set a[k] = left[i] and increase i.
    - Otherwise set a[k] = right[j] and increase j.
    - Increase k.
4. Copy the remaining elements of left (if any) into a.
5. Copy the remaining elements of right (if any) into a.

5.2 Normal Merge Sort (a, l, r)

1. If l ≥ r, return (zero or one element is already sorted).
2. Find the middle: m = l + (r − l) / 2.
3. Call MergeSort(a, l, m) to sort the left half.
4. Call MergeSort(a, m + 1, r) to sort the right half.
5. Call Merge(a, l, m, r).

5.3 Multithreaded Merge Sort (a, l, r, depth)

1. If l ≥ r, return.
2. Find the middle: m = l + (r − l) / 2.
3. If depth > 0 and the part has more than THRESHOLD elements:
    - Create a new thread T that runs ParallelMergeSort(a, l, m, depth − 1).
    - In the current thread run ParallelMergeSort(a, m + 1, r, depth − 1).
    - Call T.join() to wait for the left half.
4. Otherwise sort both halves one after another with depth = 0 (no new thread).
5. Call Merge(a, l, m, r).

5.4 Flowcharts

Normal Merge Sort

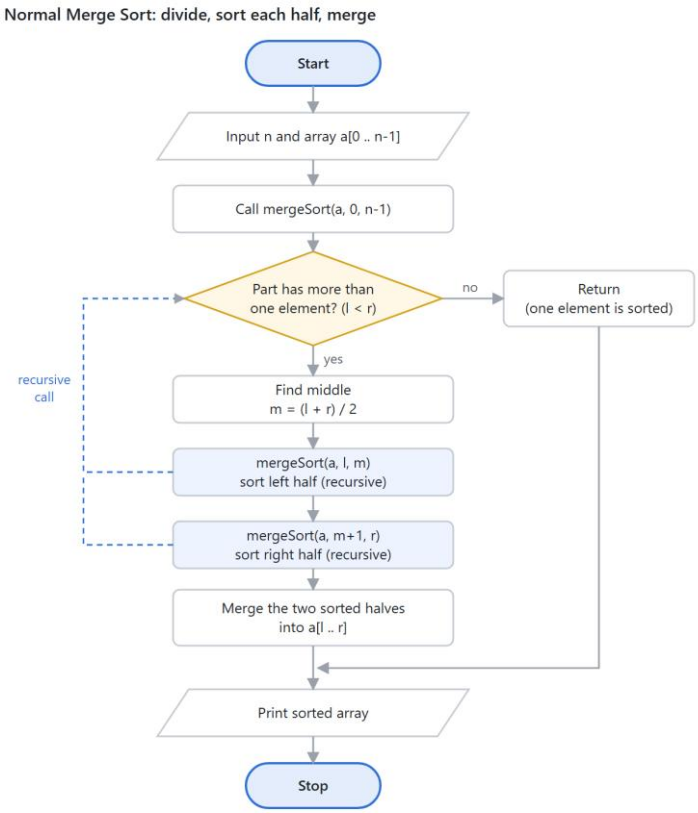


Figure 2: Normal Merge Sort flowchart · 1 decision, 2 recursive calls

The dashed lines show the recursive calls: each call goes back to the decision box with a smaller part of the array. Recursion stops when a part has only one element.

Multithreaded Merge Sort

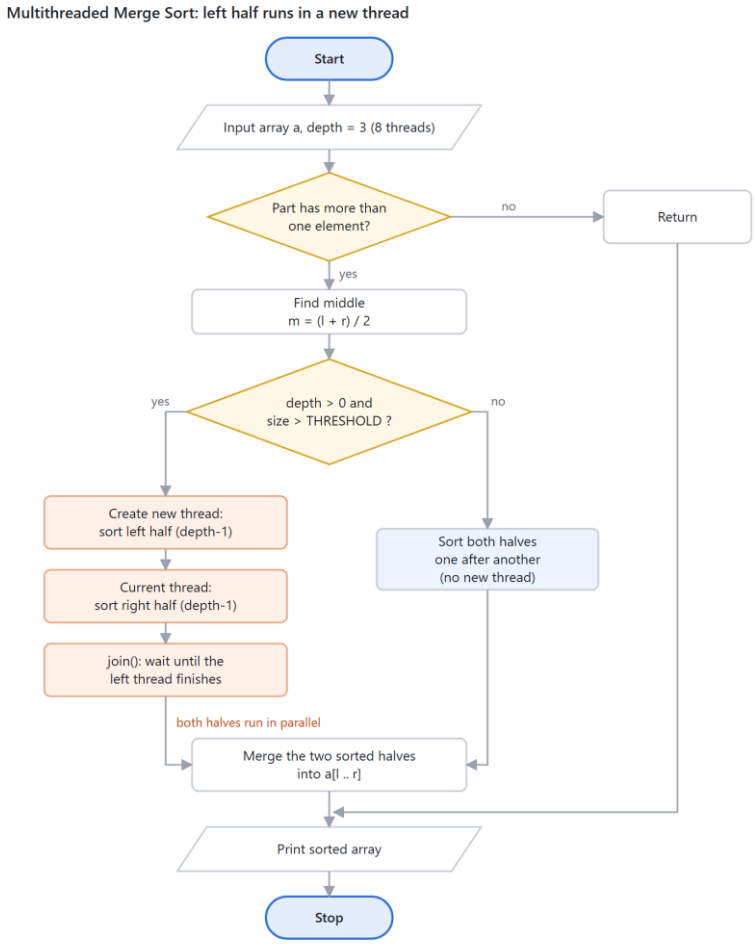


Figure 3: Multithreaded Merge Sort flowchart · 2 decisions

The orange path runs the two halves in parallel. The blue path is the same as normal Merge Sort and is used when the depth limit is reached or the part is small.

Shapes used: oval = start / stop, parallelogram = input / output, rectangle = step, diamond = decision.

6. Mathematical Model

Input: an array A of n numbers. Output: a permutation of A in which A[0] ≤ A[1] ≤ … ≤ A[n−1]. All formulas used in the project are listed below.

1. Recurrence of Merge Sort. Two halves of size n/2 are sorted, then a merge of n elements takes c·n time.

$$T(n) = 2 \cdot T(n/2) + c \cdot n, \quad T(1) = c$$

2. Solving the recurrence. The recursion tree has $\log_2 n$ levels and each level does c·n work. (Master Theorem: a = 2, b = 2, f(n) = n = n^($\log_2 2$), so Case 2.)

$$T(n) = c \cdot n \cdot \log_2 n + c \cdot n = \Theta(n \log n)$$

3. Comparisons in the best case (already sorted, n is a power of 2):

$$C_best(n) = (n/2) \cdot \log_2 n$$

4. Comparisons in the worst case (exact for any n). The program prints the same numbers, e.g. n = 1000 → 8,977 and n = 10,00,000 → 1,89,51,425.

$$C_worst(n) = n \cdot \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$$

5. Number of threads. Each level of depth doubles the number of threads that work at the same time.

$$p = 2^{depth} \quad (depth = 3 \rightarrow p = 8 \text{ threads})$$

6. Parallel running time. The p leaf parts of size n/p are sorted in parallel. The merges in the top levels are done by fewer threads, and the very last merge of n elements is done by one thread.

$$T_p(n) \approx (n/p) \cdot \log_2(n/p) + n \cdot (1 + 1/2 + \ldots + 1/2^{(depth-1)}) \leq (n/p) \cdot \log_2(n/p) + 2n$$

7. Speed-up and efficiency.

$$\text{Speed-up } S = T_normal / T_multithreaded \qquad \text{Efficiency } E = S / p$$

8. Amdahl's Law. If a fraction f of the work can run in parallel, the speed-up with p threads is limited by the part that stays serial (here, mainly the last merge).

$$S(p) \leq 1 / ((1 - f) + f / p)$$

Worked example (Test Case 1, n = 8)

A = [38, 27, 43, 3, 9, 82, 10, 1]. Level 1 merges pairs: 4 comparisons. Level 2 merges [27 38]+[3 43] and [9 82]+[1 10]: 3 + 3 = 6 comparisons. Level 3 merges [3 27 38 43]+[1 9 10 82]: 7 comparisons.

$$\text{Comparisons} = 4 + 6 + 7 = 17 = 8 \cdot 3 - 2^3 + 1 = C_worst(8)$$

So this input is a worst case for n = 8. The program also prints 17. The best case (sorted) for n = 8 would need (8/2)·3 = 12 comparisons.

Worked example (speed-up, n = 10,00,000, 8 threads)

$$S = 196.268 / 59.702 = 3.29 \qquad E = 3.29 / 8 = 0.41 \ (41\%)$$

$$3.29 = 1 / ((1 - f) + f/8) \ \Rightarrow \ f \approx 0.80, \ \text{so maximum possible } S = 1 / (1 - 0.80) \approx 5$$

7. Program Implementation

The program offers two choices. **Choice 1** reads an array from the user, sorts it with both versions, and prints the sorted arrays, times, comparisons and speed-up. **Choice 2** runs a full performance test on random, best-case, reverse-sorted and worst-case arrays of size 1,000 to 50,00,000, and then tests 1, 2, 4, 8 and 16 threads. Every time is the best of several runs to reduce noise.

```
// LP-III : Design & Analysis of Algorithms (410241) - Mini Project
// Topic 2: Merge Sort & Multithreaded Merge Sort
// Version 1 : Normal (single-threaded) Merge Sort
// Version 2 : Multithreaded Merge Sort (std::thread, depth-limited)

#include <bits/stdc++.h>
using namespace std;
using namespace chrono;

long long comparisons = 0;     // counted only by the normal merge sort

// ---------- Merge two sorted halves a[l..m] and a[m+1..r] ----------
void merge(vector<int>& a, int l, int m, int r, bool count) {
    vector<int> left(a.begin() + l, a.begin() + m + 1);
    vector<int> right(a.begin() + m + 1, a.begin() + r + 1);

    size_t i = 0, j = 0;
```

```
    int k = l;
    while (i < left.size() && j < right.size()) {
        if (count) comparisons++;
        if (left[i] <= right[j]) a[k++] = left[i++];
        else                     a[k++] = right[j++];
    }
    while (i < left.size())  a[k++] = left[i++];    // copy what is left
    while (j < right.size()) a[k++] = right[j++];
}

// ---------- 1. Normal Merge Sort ----------
void mergeSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r, true);
}

// ---------- 2. Multithreaded Merge Sort ----------
// depth = how many more levels may create a new thread.
// depth 3 -> up to 2^3 = 8 threads work at the same time.
const int THRESHOLD = 1000;    // small parts are sorted without a new thread

void parallelMergeSort(vector<int>& a, int l, int r, int depth) {
    if (l >= r) return;
    int m = l + (r - l) / 2;

    if (depth > 0 && (r - l + 1) > THRESHOLD) {
        // left half in a new thread, right half in this thread
        thread t(parallelMergeSort, ref(a), l, m, depth - 1);
        parallelMergeSort(a, m + 1, r, depth - 1);
        t.join();                               // wait for the left half
    } else {
        parallelMergeSort(a, l, m, 0);          // no more threads: normal recursion
        parallelMergeSort(a, m + 1, r, 0);
    }
    merge(a, l, m, r, false);
}

// ---------- Input generators ----------
vector<int> randomArray(int n) {
    mt19937 gen(42);                            // fixed seed so results can be repeated
    uniform_int_distribution<int> dist(1, 1000000);
    vector<int> a(n);
    for (int& x : a) x = dist(gen);
    return a;
}

vector<int> sortedArray(int n) {                // best case: fewest comparisons
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = i + 1;
    return a;
}

vector<int> reverseArray(int n) {
    vector<int> a(n);
    for (int i = 0; i < n; i++) a[i] = n - i;
    return a;
}

// Worst case: every merge must compare until both halves are empty.
// Build it by "un-merging" a sorted array: even positions go left, odd go right.
void makeWorst(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    vector<int> left, right;
```

```
    for (int i = l; i <= r; i++) {
        if ((i - l) % 2 == 0) left.push_back(a[i]);
        else                  right.push_back(a[i]);
    }
    int k = l;
    for (int x : left)  a[k++] = x;
    for (int x : right) a[k++] = x;
    makeWorst(a, l, m);
    makeWorst(a, m + 1, r);
}

vector<int> worstArray(int n) {
    vector<int> a = sortedArray(n);
    makeWorst(a, 0, n - 1);
    return a;
}

// ---------- Helpers ----------
void printArray(const vector<int>& a) {
    for (int x : a) cout << x << " ";
    cout << "\n";
}

int threadsUsed(int depth) { return 1 << depth; }

// Runs both versions on the same input and prints time, speed-up, comparisons.
void runBoth(const vector<int>& input, int depth, bool show) {
    vector<int> a1, a2;
    int n = input.size();

    // run 5 times and keep the best time (first run is slow due to cache warm-up)
    double sTime = DBL_MAX, pTime = DBL_MAX;
    for (int rep = 0; rep < 5; rep++) {
        a1 = input; a2 = input; comparisons = 0;
        auto t1 = high_resolution_clock::now();
        mergeSort(a1, 0, n - 1);
        auto t2 = high_resolution_clock::now();
        parallelMergeSort(a2, 0, n - 1, depth);
        auto t3 = high_resolution_clock::now();
        sTime = min(sTime, duration<double, milli>(t2 - t1).count());
        pTime = min(pTime, duration<double, milli>(t3 - t2).count());
    }

    if (show) {
        cout << "\nSorted (normal)        : "; printArray(a1);
        cout << "Sorted (multithreaded) : "; printArray(a2);
    }
    cout << fixed << setprecision(4);
    cout << "Normal Merge Sort time        (ms) : " << sTime << "\n";
    cout << "Multithreaded Merge Sort time (ms) : " << pTime
         << "  [up to " << threadsUsed(depth) << " threads]\n";
    cout << "Comparisons (normal)               : " << comparisons << "\n";
    cout << setprecision(2);
    if (pTime > 0) cout << "Speed-up (normal / multithreaded)  : " << sTime / pTime << "\n";
    else           cout << "Speed-up (normal / multithreaded)  : too fast to measure\n";
    if (n <= THRESHOLD)
        cout << "Note: n <= THRESHOLD (" << THRESHOLD
             << "), so no extra thread is created for this small array.\n";
    cout << "Both results same and sorted?      : "
         << ((a1 == a2 && is_sorted(a1.begin(), a1.end())) ? "YES" : "NO") << "\n";
}

void benchmark(int depth) {
    int sizes[] = {1000, 10000, 100000, 1000000, 5000000};
    string names[] = {"Random", "Best (sorted)", "Reverse sorted", "Worst case"};

    cout << "\nCPU cores available : " << thread::hardware_concurrency()
```

```
         << " | Threads used : " << threadsUsed(depth) << "\n";
    cout << left << setw(10) << "n" << setw(16) << "Input"
         << setw(14) << "Comparisons" << setw(14) << "Normal(ms)"
         << setw(14) << "Thread(ms)" << "Speed-up\n";
    cout << string(76, '-') << "\n";

    for (int n : sizes) {
        for (int c = 0; c < 4; c++) {
            vector<int> in = (c == 0) ? randomArray(n) : (c == 1) ? sortedArray(n)
                           : (c == 2) ? reverseArray(n) : worstArray(n);
            vector<int> a1 = in, a2 = in;

            // run 3 times and keep the best time to reduce noise
            double sBest = DBL_MAX, pBest = DBL_MAX;
            long long cmp = 0;
            for (int rep = 0; rep < 3; rep++) {
                a1 = in; a2 = in; comparisons = 0;
                auto t1 = high_resolution_clock::now();
                mergeSort(a1, 0, n - 1);
                auto t2 = high_resolution_clock::now();
                parallelMergeSort(a2, 0, n - 1, depth);
                auto t3 = high_resolution_clock::now();
                sBest = min(sBest, duration<double, milli>(t2 - t1).count());
                pBest = min(pBest, duration<double, milli>(t3 - t2).count());
                cmp = comparisons;
            }
            if (a1 != a2) cout << "ERROR: results differ!\n";

            cout << left << setw(10) << n << setw(16) << names[c]
                 << setw(14) << cmp << fixed << setprecision(3)
                 << setw(14) << sBest << setw(14) << pBest
                 << setprecision(2) << sBest / pBest << "\n";
        }
    }

    // Effect of number of threads (n = 1,000,000 random)
    int n = 1000000;
    vector<int> in = randomArray(n);
    cout << "\nEffect of thread count (n = " << n << ", random input)\n";
    cout << left << setw(10) << "Threads" << setw(14) << "Time(ms)" << "Speed-up\n";
    cout << string(32, '-') << "\n";
    double base = 0;
    for (int d = 0; d <= 4; d++) {
        double best = DBL_MAX;
        for (int rep = 0; rep < 3; rep++) {
            vector<int> a = in;
            auto t1 = high_resolution_clock::now();
            parallelMergeSort(a, 0, n - 1, d);
            auto t2 = high_resolution_clock::now();
            best = min(best, duration<double, milli>(t2 - t1).count());
        }
        if (d == 0) base = best;
        cout << left << setw(10) << threadsUsed(d) << fixed << setprecision(3)
             << setw(14) << best << setprecision(2) << base / best << "\n";
    }
}

int main() {
    cout << "===== Merge Sort vs Multithreaded Merge Sort =====\n";
    cout << "1. Enter your own array\n";
    cout << "2. Run performance test (random / best / reverse / worst inputs)\n";
    cout << "Enter choice: ";
    int choice;
    cin >> choice;

    int depth = 3;                              // 2^3 = 8 threads
```

```
    if (choice == 1) {
        int n;
        cout << "Enter number of elements: ";
        cin >> n;
        if (n < 1) { cout << "At least 1 element is required.\n"; return 0; }
        vector<int> a(n);
        cout << "Enter " << n << " elements:\n";
        for (int& x : a) cin >> x;
        runBoth(a, depth, true);
    } else if (choice == 2) {
        benchmark(depth);
    } else {
        cout << "Invalid choice.\n";
    }
    return 0;
}
```

What each function does

Function	What it does (simple words)
`merge`	Merges two sorted halves a[l..m] and a[m+1..r]. Counts comparisons for the normal version.
`mergeSort`	Normal Merge Sort: splits at the middle, sorts both halves recursively, merges.
`parallelMergeSort`	Same as mergeSort, but sorts the left half in a new `std::thread` while the current thread sorts the right half, then `join()` and merge. Stops creating threads when depth = 0 or the part is ≤ THRESHOLD.
`randomArray`	Makes n random numbers (1 to 10,00,000) with a fixed seed so results can be repeated.
`sortedArray` / `reverseArray`	Make an already sorted array (best case) and a reverse sorted array.
`makeWorst` / `worstArray`	Builds the worst-case input by "un-merging" a sorted array: even positions go left, odd positions go right, recursively.
`runBoth`	Runs both versions on the user's array, measures time with `chrono`, prints results and checks that both outputs are equal and sorted.
`benchmark`	Runs both versions on all sizes and input types, prints a table, and tests different thread counts.
`main`	Shows the menu and calls runBoth or benchmark.

How to compile and run

```
g++ -std=c++17 -O2 merge_sort_comparison.cpp -o msort -pthread
./msort          (Linux)      or      msort.exe      (Windows)
```

The `-pthread` flag is needed for `std::thread` on Linux and MinGW. `-O2` turns on compiler optimisation so the timing is realistic.

8. Test Cases & Test Data

Tests 1 to 7 use choice 1 (user input). Test 8 uses choice 2 (performance test). For choice 1 the input is: the choice, then n, then the n elements.

Test	Input	Purpose	Expected result
1	8 elements: 38 27 43 3 9 82 10 1	Textbook example (also a worst case for n = 8)	1 3 9 10 27 38 43 82, 17 comparisons

Test	Input	Purpose	Expected result
2	10 elements: 10 9 8 … 1	Reverse sorted input	1 2 … 10
3	9 elements: 5 3 5 1 3 9 1 5 3	Duplicate values (stability)	1 1 3 3 3 5 5 5 9
4	7 elements: -4 10 0 -15 7 3 -1	Negative numbers and zero	-15 -4 -1 0 3 7 10
5	1 element: 42	Smallest valid input	42, 0 comparisons
6	n = 0	Invalid size	Error message
7	Menu choice 3	Invalid menu choice	"Invalid choice."
8	n = 10^3, 10^4, 10^5, 10^6, 5×10^6 × 4 input types	Performance: best, worst, reverse, random	Same sorted output from both; time table

Test 1 input

```
1
8
38 27 43 3 9 82 10 1
```

Test 2 input

```
1
10
10 9 8 7 6 5 4 3 2 1
```

Test 3 input

```
1
9
5 3 5 1 3 9 1 5 3
```

Test 4 input

```
1
7
-4 10 0 -15 7 3 -1
```

Test 5, 6, 7 input

```
1
1
42

1
0

3
```

Test 8 input

```
2
```

The program builds the large arrays by itself: random (fixed seed 42), sorted 1..n (best case), reverse n..1, and the worst-case pattern.

9. Sample Outputs

Test 1 output

```
===== Merge Sort vs Multithreaded Merge Sort =====
1. Enter your own array
2. Run performance test (random / best / reverse / worst inputs)
Enter choice: 1
Enter number of elements: 8
Enter 8 elements:
38 27 43 3 9 82 10 1
```

```
Sorted (normal)        : 1 3 9 10 27 38 43 82
Sorted (multithreaded) : 1 3 9 10 27 38 43 82
Normal Merge Sort time        (ms) : 0.0009
Multithreaded Merge Sort time (ms) : 0.0009  [up to 8 threads]
Comparisons (normal)               : 17
Speed-up (normal / multithreaded)  : 1.00
Note: n <= THRESHOLD (1000), so no extra thread is created for this small array.
Both results same and sorted?      : YES
```

Manual check: 4 + 6 + 7 = **17** comparisons (Section 6). For 8 elements no thread is created, so both times are the same.

Test 2 output

```
Sorted (normal)        : 1 2 3 4 5 6 7 8 9 10
Sorted (multithreaded) : 1 2 3 4 5 6 7 8 9 10
Normal Merge Sort time        (ms) : 0.0005
Multithreaded Merge Sort time (ms) : 0.0006  [up to 8 threads]
Comparisons (normal)               : 15
Speed-up (normal / multithreaded)  : 0.83
Note: n <= THRESHOLD (1000), so no extra thread is created for this small array.
Both results same and sorted?      : YES
```

Reverse input needs only 15 comparisons. The worst case for n = 10 would be 10·4 − 16 + 1 = 25.

Test 3 output

```
Sorted (normal)        : 1 1 3 3 3 5 5 5 9
Sorted (multithreaded) : 1 1 3 3 3 5 5 5 9
Comparisons (normal)               : 19
Both results same and sorted?      : YES
```

Test 4 output

```
Sorted (normal)        : -15 -4 -1 0 3 7 10
Sorted (multithreaded) : -15 -4 -1 0 3 7 10
Comparisons (normal)               : 13
Both results same and sorted?      : YES
```

Test 5, 6, 7 output

```
Sorted (normal)        : 42
Sorted (multithreaded) : 42
Comparisons (normal)               : 0
Speed-up (normal / multithreaded)  : too fast to measure

Enter number of elements: At least 1 element is required.

Enter choice: Invalid choice.
```

Test 8 output (performance test)

```
Enter choice: 2
CPU cores available : 12 | Threads used : 8
n         Input           Comparisons   Normal(ms)    Thread(ms)    Speed-up
------------------------------------------------------------------------------
1000      Random          8690          0.286         0.299         0.96
1000      Best (sorted)   5044          0.221         0.219         1.01
1000      Reverse sorted  4932          0.210         0.208         1.01
1000      Worst case      8977          0.220         0.205         1.07
10000     Random          120465        3.073         2.742         1.12
10000     Best (sorted)   69008         1.530         2.778         0.55
10000     Reverse sorted  64608         1.167         1.563         0.75
10000     Worst case      123617        1.214         1.565         0.78
100000    Random          1536424       12.442        4.487         2.77
100000    Best (sorted)   853904        7.478         2.847         2.63
100000    Reverse sorted  815024        7.688         3.287         2.34
100000    Worst case      1568929       8.529         3.049         2.80
1000000   Random          18675342      145.239       39.850        3.64
```

```
1000000   Best (sorted)   10066432      84.478        25.171        3.36
1000000   Reverse sorted  9884992       79.674        22.978        3.47
1000000   Worst case      18951425      95.318        24.679        3.86
5000000   Random          105051246     790.968       219.307       3.61
5000000   Best (sorted)   56894080      459.432       139.718       3.29
5000000   Reverse sorted  54717312      730.614       203.810       3.58
5000000   Worst case      106611393     738.539       168.658       4.38

Effect of thread count (n = 1000000, random input)
Threads   Time(ms)      Speed-up
-------------------------------
1         196.268       1.00
2         109.582       1.79
4         84.251        2.33
8         59.702        3.29
16        67.748        2.90
```

Times depend on the computer and on other programs running, so your values will differ, but the pattern will be the same.

10. Time & Space Complexity Analysis

10.1 Normal Merge Sort

- The array is halved $\log_2 n$ times, so there are $\log_2 n$ levels of recursion.
- Each level merges n elements in total, which takes O(n) time.
- **Time Complexity = O(n log n)** in the best, average and worst case. Only the number of comparisons changes: $(n/2)\cdot\log_2 n$ in the best case and about $n\cdot\log_2 n$ in the worst case.
- The merge step uses temporary arrays of total size n, and the recursion stack is $\log_2 n$ deep.
- **Space Complexity = O(n)** (auxiliary array) + O(log n) (stack) = **O(n)**.

10.2 Multithreaded Merge Sort

- **Total work** (all threads together) is still O(n log n), the same as the normal version, plus O(p) for creating p threads.
- **Time with p threads:** $O((n/p)\cdot\log(n/p) + n)$. The p leaf parts are sorted in parallel; the top merges are done by fewer threads, and the final merge of n elements uses only one thread.
- With unlimited threads the time becomes O(n), because the final merge is serial. So the best possible speed-up is O(log n). (A parallel merge would be needed to do better.)
- **Space Complexity = O(n)** for the merge arrays + O(log n) stack in each thread + O(p) for thread objects and their stacks = **O(n)**.

10.3 Summary

Algorithm	Best case	Worst case	Space	Stable?
Normal Merge Sort	O(n log n)	O(n log n)	O(n)	Yes
Multithreaded Merge Sort (p threads)	O((n/p) log(n/p) + n)	O((n/p) log(n/p) + n)	O(n + p)	Yes

Input type	Comparisons (formula)	n = 10,00,000 (from program)
Best case (sorted)	$\approx (n/2)\cdot\log_2 n$	1,00,66,432
Reverse sorted	$\approx (n/2)\cdot\log_2 n$	98,84,992
Random	$\approx n\cdot\log_2 n - 1.25n$	1,86,75,342

Input type	Comparisons (formula)	n = 10,00,000 (from program)
Worst case	$n \cdot \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$	1,89,51,425 (exact match)

11. Performance Comparison

With 8 threads, the multithreaded version was **3.3 to 4.4 times faster** for 10 lakh and 50 lakh elements. For 10,000 elements or fewer it was about the same or even **slower**, because creating threads costs more than the work saved. The table is taken from the Test 8 output in Section 9 (time in ms, best of 3 runs).

n	Input	Comparisons	Normal (ms)	Threaded (ms)	Speed-up
1,000	Random	8,690	0.286	0.299	0.96
1,000	Best (sorted)	5,044	0.221	0.219	1.01
1,000	Reverse sorted	4,932	0.210	0.208	1.01
1,000	Worst case	8,977	0.220	0.205	1.07
10,000	Random	1,20,465	3.073	2.742	1.12
10,000	Best (sorted)	69,008	1.530	2.778	0.55
10,000	Reverse sorted	64,608	1.167	1.563	0.75
10,000	Worst case	1,23,617	1.214	1.565	0.78
1,00,000	Random	15,36,424	12.442	4.487	2.77
1,00,000	Best (sorted)	8,53,904	7.478	2.847	2.63
1,00,000	Reverse sorted	8,15,024	7.688	3.287	2.34
1,00,000	Worst case	15,68,929	8.529	3.049	2.80
10,00,000	Random	1,86,75,342	145.239	39.850	3.64
10,00,000	Best (sorted)	1,00,66,432	84.478	25.171	3.36
10,00,000	Reverse sorted	98,84,992	79.674	22.978	3.47
10,00,000	Worst case	1,89,51,425	95.318	24.679	3.86
50,00,000	Random	10,50,51,246	790.968	219.307	3.61
50,00,000	Best (sorted)	5,68,94,080	459.432	139.718	3.29
50,00,000	Reverse sorted	5,47,17,312	730.614	203.810	3.58
50,00,000	Worst case	10,66,11,393	738.539	168.658	4.38

Rows in orange: the multithreaded version was slower (speed-up < 1).

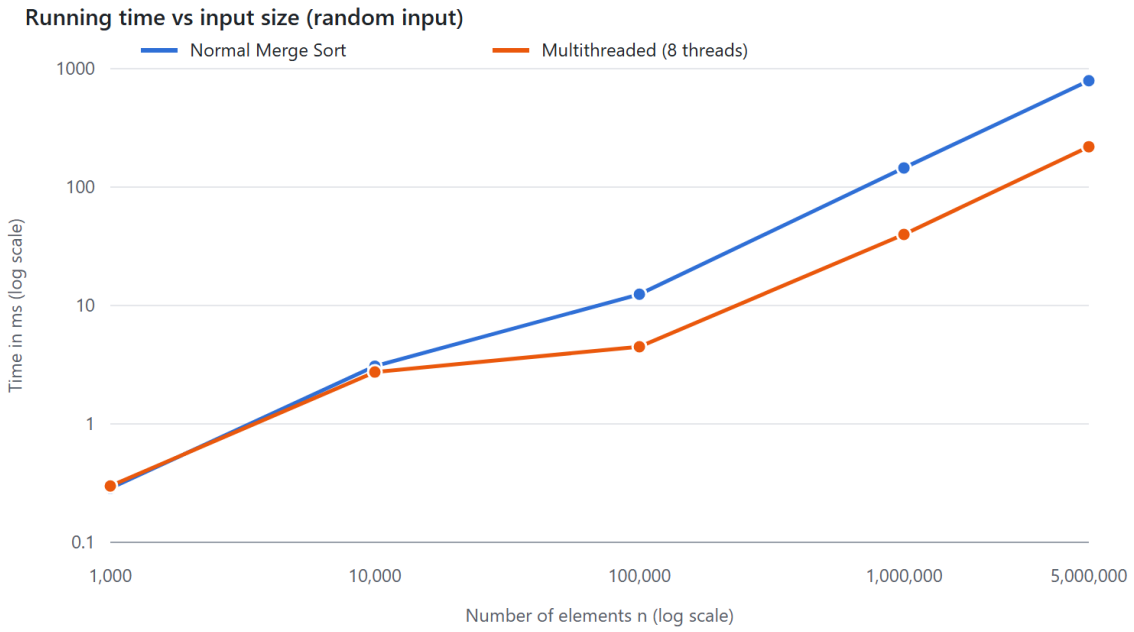


Figure 4: Running time of both versions for random input (both axes on log scale)

Effect of the number of threads (n = 10,00,000, random input)

Threads (p)	Time (ms)	Speed-up S	Efficiency E = S/p
1	196.268	1.00	100%
2	109.582	1.79	90%
4	84.251	2.33	58%
8	59.702	3.29	41%
16	67.748	2.90	18%

Here the speed-up is measured against the same parallelMergeSort function run with 1 thread (depth = 0) in a separate run, so the times differ slightly from the main table.

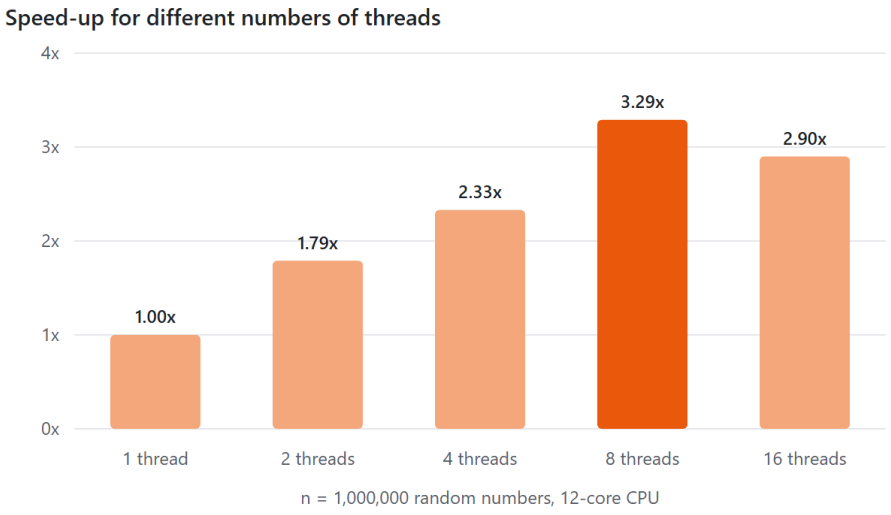


Figure 5: Speed-up for 1 to 16 threads

Observations

- **Small inputs (n ≤ 10,000):** speed-up between 0.55 and 1.12. The work is less than a millisecond, so the cost of creating threads and waiting for them hides any gain.
- **Large inputs (n ≥ 1,00,000):** speed-up grows with n: about 2.6× at 1 lakh, 3.5× at 10 lakh and up to 4.4× at 50 lakh.
- **More threads are not always better.** Speed-up rose up to 8 threads (3.29×) and then fell at 16 threads (2.90×), because the CPU has only 12 hardware threads and the extra threads only add switching cost.
- **Why not 8× with 8 threads?** (1) The final merge of all n elements runs on one thread (Amdahl's Law, f ≈ 0.80 gives at most about 5×). (2) The i5-1235U has only 2 fast performance cores; the other 8 cores are slower. (3) All threads share the same memory bandwidth.
- **Best vs worst case:** the sorted (best case) input needs about half the comparisons of the worst case (1.0 crore vs 1.9 crore at n = 10 lakh) and was faster (84 ms vs 95 ms). Both versions still grow as n log n in every case.
- **Random input was the slowest** even though the worst case makes slightly more comparisons, because random data makes the CPU's branch prediction fail often. Sorted, reverse and worst-case inputs follow a regular pattern that the CPU predicts well.

Point	Normal Merge Sort	Multithreaded Merge Sort
Threads used	1	Up to 2^depth (8 in this project)
Total work	O(n log n)	O(n log n) + thread creation cost
Running time	O(n log n)	O((n/p) log(n/p) + n)
Small input (≤ 10,000)	Faster or equal	Same or slower (thread overhead)
Large input (≥ 1,00,000)	Slower	2.3× to 4.4× faster in our tests
Code complexity	Simple	Needs thread creation, join, depth limit
Best used for	Small arrays, single-core systems	Large arrays on multi-core CPUs

12. Conclusion

- Merge Sort is a Divide and Conquer algorithm that always runs in **O(n log n)** time and O(n) extra space. The best case (sorted input) needs about half the comparisons of the worst case, but the order of growth is the same.
- Because the two halves do not overlap, they can be sorted on different threads without locks. The multithreaded version gave the **same sorted output** in every test.
- Multithreading gave a speed-up of **3.3× to 4.4×** for large arrays (10 lakh and 50 lakh elements) with 8 threads, but was **slower for small arrays** because of thread creation cost.
- The speed-up is limited by the serial final merge (Amdahl's Law), the number of CPU cores, and memory bandwidth. Using more threads than hardware threads made it slower.
- Use the normal version for small inputs and the multithreaded version, with a depth limit and threshold, for large inputs on multi-core machines.
- **Future scope:** parallel merge (to remove the serial final merge), a thread pool instead of creating new threads, OpenMP tasks, and using Insertion Sort for very small parts.

13. References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms* (Chapter 2: Merge Sort; Chapter 27: Multithreaded Algorithms), MIT Press.

2. E. Horowitz, S. Sahni, S. Rajasekaran, *Fundamentals of Computer Algorithms*, Universities Press.
3. A. Levitin, *Introduction to the Design and Analysis of Algorithms*, Pearson.
4. A. Williams, *C++ Concurrency in Action*, Manning Publications.
5. cppreference.com, std::thread and <chrono>: https://en.cppreference.com
6. GeeksforGeeks, "Merge Sort": https://www.geeksforgeeks.org
7. SPPU BE Computer Engineering syllabus, LP-III (410241).